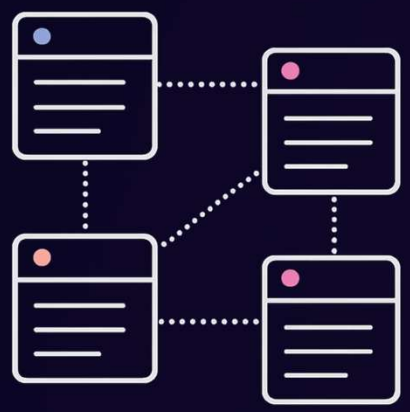
SQL Project on Online Bookstore

End-to-End Basic & Advanced SQL Queries for Real-World Data Analysis

Mastering SQL for E-commerce Insights

Project Overview



Three CSV Files

Books.csv • Customers.csv • Orders.csv

Common Keys

Book_ID and Customer_ID enable seamless table joins

Project Objective

Practice real-world SQL queries on an online bookstore database — from simple retrieval to complex analytics

Database Schema

Books Table

```
CREATE TABLE Books (
  Book_ID INT PRIMARY KEY,
  Title VARCHAR(255),
  Author VARCHAR(255),
  Genre VARCHAR(100),
  Published_Year INT,
  Price DECIMAL(10,2),
  Stock INT
);
```

Customers Table

```
CREATE TABLE Customers (
  Customer_ID INT PRIMARY
KEY,
  Name VARCHAR(255),
  Email VARCHAR(255),
  Phone VARCHAR(20),
  City VARCHAR(100),
  Country VARCHAR(100)
);
```

Orders Table

```
CREATE TABLE Orders (
 Order_ID INT PRIMARY KEY,
 Customer_ID INT,
 Book_ID INT,
 Order_Date DATE,
 Quantity INT,
 Total_Amount
DECIMAL(10,2),
 FOREIGN KEY (Customer_ID)
REFERENCES
Customers(Customer_ID),
 FOREIGN KEY (Book_ID)
REFERENCES Books(Book_ID)
);
```

Basic Queries – Part 1

01

Retrieve all books in the "Fiction" genre

```
SELECT * FROM Books
WHERE Genre = 'Fiction';
```

Filter books by specific genre category

02

Find books published after 1950

```
SELECT * FROM Books
WHERE Published_Year > 1950;
```

Identify modern publications

03

List all customers from Canada

```
SELECT * FROM Customers
WHERE Country = 'Canada';
```

Target customers by geographic location

04

Show orders placed in November 2023

```
SELECT * FROM Orders
WHERE Order_Date LIKE '2023-11-%';
```

Analyse seasonal ordering patterns

Basic Queries – Part 2

Query 5: Total Stock

```
SELECT SUM(Stock) AS Total_Stock
FROM Books;
```

Calculate inventory availability

Query 6: Most Expensive Book

```
SELECT * FROM Books
ORDER BY Price DESC
LIMIT 1;
```

Identify premium products

Query 7: Bulk Orders

```
SELECT c.*, o.*
FROM Customers c
JOIN Orders o ON c.Customer_ID = o.Customer_ID
WHERE o.Quantity > 1;
```

Find customers ordering multiple copies

Query 8: High-Value Orders

```
SELECT * FROM Orders
WHERE Total_Amount > 20;
```

Filter orders exceeding threshold

Basic Queries – Part 3

Query 9: Available Genres

```
SELECT DISTINCT Genre
FROM Books;
```

Explore genre diversity

Query 10: Low Stock Alert

```
SELECT * FROM Books
ORDER BY Stock ASC
LIMIT 1;
```

Identify inventory shortages

Query 11: Total Revenue

```
SELECT SUM(Total_Amount)
AS Revenue
FROM Orders;
```

Calculate overall sales

Advanced Queries – Part 1

Books Sold by Genre

```
SELECT b.Genre,
SUM(o.Quantity) AS
Total_Sold
FROM Books b
JOIN Orders o ON b.Book_ID
= o.Book_ID
GROUP BY b.Genre;
```

Analyse genre popularity

Average Fantasy Price

```
SELECT AVG(Price) AS
Avg_Price
FROM Books
WHERE Genre = 'Fantasy';
```

Understand pricing strategy

Frequent Customers

```
SELECT c.Customer_ID,
c.Name, COUNT(o.Order_ID)
AS Order_Count
FROM Customers c
JOIN Orders o ON
c.Customer_ID =
o.Customer_ID
GROUP BY c.Customer_ID
HAVING COUNT(o.Order_ID) >=
2;
```

Identify loyal customers

Advanced Queries – Part 2

1

Most Frequently Ordered Book

```
SELECT b.Title,
COUNT(o.Order_ID) AS
Order_Frequency
FROM Books b
JOIN Orders o ON b.Book_ID =
o.Book_ID
GROUP BY b.Book_ID
ORDER BY Order_Frequency DESC
LIMIT 1;
```

Determine top-selling titles

2

Top 3 Expensive Fantasy Books

```
SELECT * FROM Books
WHERE Genre = 'Fantasy'
ORDER BY Price DESC
LIMIT 3;
```

Highlight premium fantasy offerings

3

Total Books Sold by Author

```
SELECT b.Author,
SUM(o.Quantity) AS
Total_Quantity
FROM Books b
JOIN Orders o ON b.Book_ID =
o.Book_ID
GROUP BY b.Author;
```

Analyse author performance

Advanced Queries – Part 3

Query 7: High-Spending Cities

```
SELECT DISTINCT c.City
FROM Customers c
JOIN Orders o ON
c.Customer_ID =
o.Customer_ID
GROUP BY c.City,
c.Customer_ID
HAVING SUM(o.Total_Amount)
> 30;
```

Geographic targeting for marketing

Query 8: Top Customer

```
SELECT c.Customer_ID,
c.Name,
SUM(o.Total_Amount) AS
Total_Spent
FROM Customers c
JOIN Orders o ON
c.Customer_ID =
o.Customer_ID
GROUP BY c.Customer_ID
ORDER BY Total_Spent DESC
LIMIT 1;
```

Identify highest-value customer

Query 9: Remaining Stock

```
SELECT b.Book_ID, b.Title,
b.Stock -
COALESCE(SUM(o.Quantity),
0) AS Remaining_Stock
FROM Books b
LEFT JOIN Orders o ON
b.Book_ID = o.Book_ID
GROUP BY b.Book_ID,
b.Title, b.Stock;
```

Calculate post-fulfilment inventory

Key Takeaways & Next Steps

Mastered 20 Practical Queries

From basic SELECT statements to advanced JOINs, GROUP BY operations, and business analytics

Complete Relational Model

Built comprehensive database structure for e-commerce bookstore with proper foreign key relationships

Real-World Application

Ready to apply these skills to business intelligence and data-driven decision making

---

Which query would you use first in your next project?

Share your thoughts in the comments below 👇